Fundamentos de Aprendizagem Automática

Class 02 — Blind vs. Gradient-Based Optimization

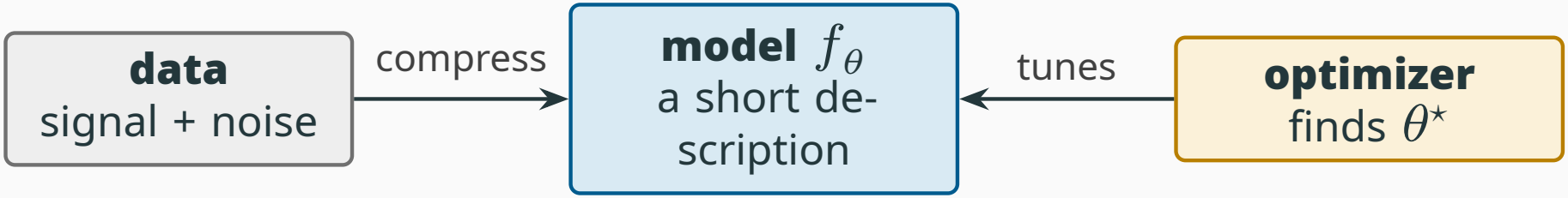
Mário Antunes

Universidade de Aveiro

2026

From Compression to Optimization

Where We Left Off



Class 01: *what* a good model is (MDL, evaluation)
Class 02: *how* to find its parameters

- **Signal and noise:** the data hide a pattern behind noise
- **Learning = compression:** a good model keeps the pattern in few numbers
- **Compression needs optimization:** the numbers θ must be *found*

The Loss Is the Code Length

$$\theta^\star = \arg\min_\theta \underbrace{\mathcal{L}_{data}(D;\theta)}_{\approx L(D|H)} + \lambda \underbrace{\Omega(\theta)}_{\approx L(H)}$$

- The two-part MDL code from Class 01, written as a **loss** to minimize
- Every model in this course is trained by minimizing a loss like this one
- The model family fixes the *shape* of $\mathcal{L}$; the optimizer walks on it

What Is Optimization?

The Optimization Problem

$$\min_{x \in \mathcal{X}} \; f(x) \qquad \text{subject to} \quad g_i(x) \leq 0, \quad h_j(x) = 0$$

- x: the **decision variables** (in ML: the parameters θ)
- f: the **objective** (in ML: the loss $\mathcal{L}$)
- $\mathcal{X}$ with g_i, h_j: the **feasible set** (e.g. box bounds $\ell \leq x \leq u$)
- Maximizing is the same problem: $\max f = -\min(-f)$
- $x^\star = \arg\min f$ is the **minimizer**; $f(x^\star)$ is the **minimum**

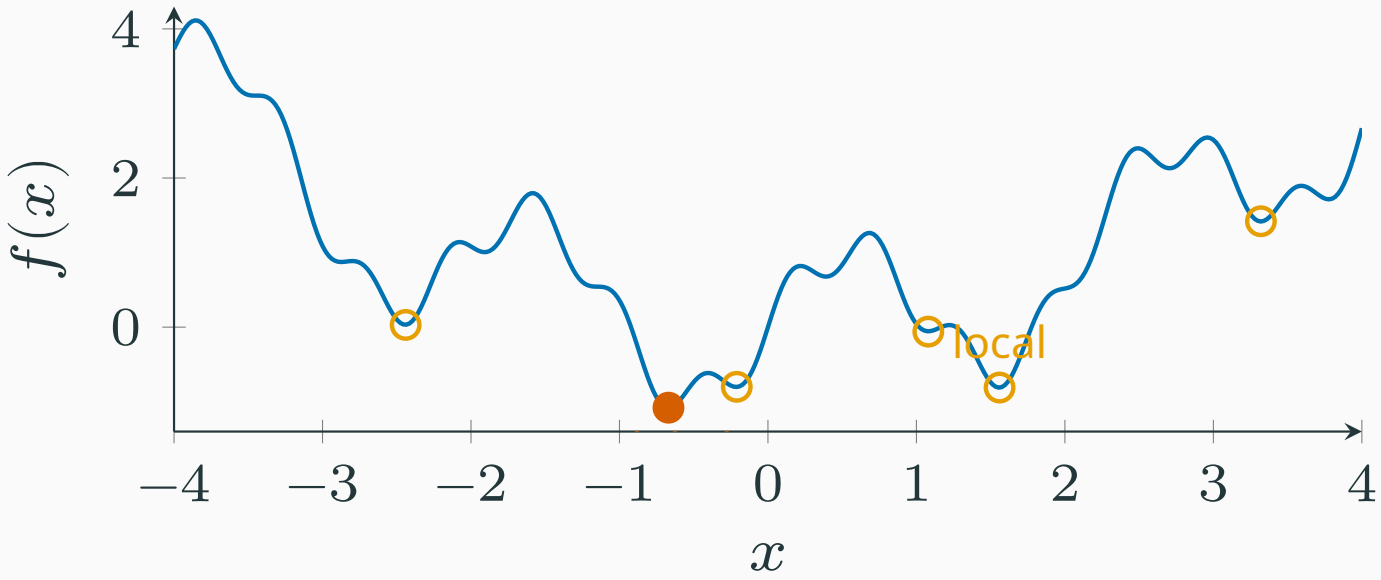
A Numerical Example

Fit a line $\hat{y} = w\,x + b$ to three points: $(0, 1)$, $(1, 2)$, $(2, 4)$

Concept	In this example
decision variables	$\theta = (w, b)$
objective	$\mathcal{L}(w, b) = \frac{1}{3}\sum_{i=1}^{3}(wx_i + b - y_i)^2$
feasible set	the box $-5 \le w \le 5,\ -5 \le b \le 5$
one guess	$\theta = (1, 1)$: errors $0, 0, -1$, so $\mathcal{L} = 1/3 = 0.333$
minimizer	$\theta^\star = (1.5,\ 0.833)$
minimum	$\mathcal{L}(\theta^\star) = 0.056$

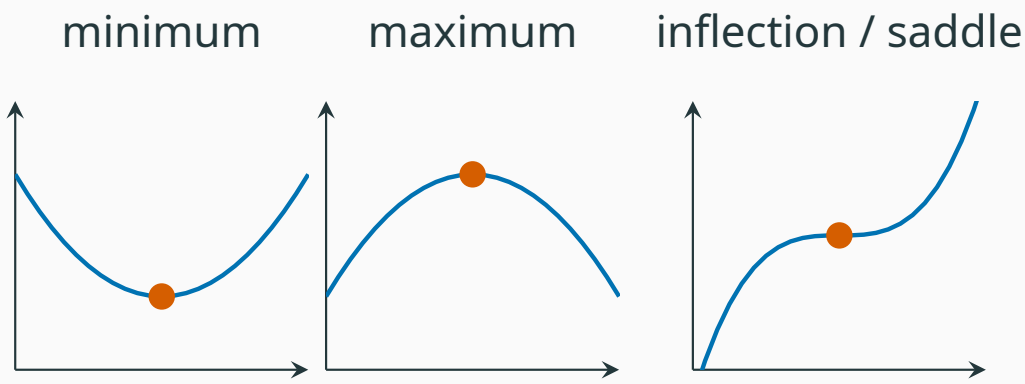
- Constraint as $g(x) \le 0$: e.g. $g_1(\theta) = w - 5 \le 0$
- **Maximizing** accuracy is the same as **minimizing** the error rate $1 -$ accuracy

Global and Local Minima



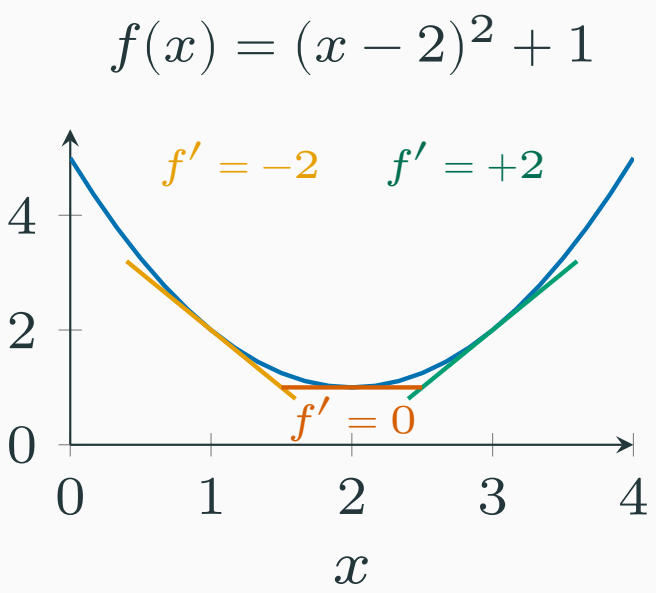
- **Global minimum:** $f(x^\star) \leq f(x)$ for **every** feasible x
- **Local minimum:** $f(x^\star) \leq f(x)$ for every x **near** $x^\star$
- Most optimizers only *guarantee* a local minimum

Optimality Conditions



- **First order:** at an interior minimum the slope vanishes, $f'(x^\star) = 0$ (in n-D: $\nabla f(x^\star) = 0$)
- **Second order:** $f''(x^\star) > 0$ (in n-D: the Hessian H is positive definite)
- $f' = 0$ alone also holds at maxima and saddles: a **stationary point** is not always a minimum

What the Second Derivative Tells Us

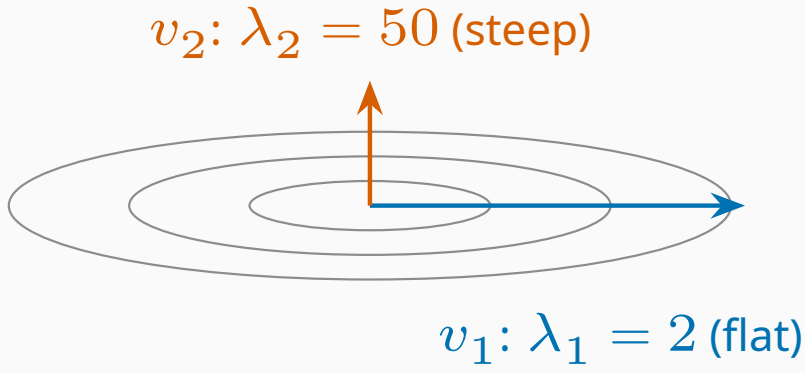


- Through the minimum, f' goes from **negative** to **positive**
- $f'' = 2 > 0$: the slope **increases**, the curve bends **up** (a cup)
- $f'' < 0$: a cap (maximum); $f'' = 0$: no verdict (x^3 at 0)

- f'' is the **curvature**: it says how far the bottom is, $\delta = -f'/f''$, not only which way
- **Newton's method** uses it: few steps, no learning rate, very stable near a minimum
- The price: f must be **smooth** (twice differentiable); on rugged or noisy losses curvature misleads

Vectors and Matrices: What We Need

- **Vector** $v \in \mathbb{R}^n$: a point, a direction, a gradient
- **Matrix** $H \in \mathbb{R}^{n \times n}$; **symmetric** if $H = H^\top$
- **Eigenpair:** $Hv = \lambda v$; H only stretches v by λ
- Hessian: λ_i = **curvature** along direction v_i
- **Positive definite:** all $\lambda_i > 0$
- **Condition number** $\kappa = \lambda_{max} / \lambda_{min}$
- Solve $H\delta = -g$; never compute H^{-1}



$$H = \begin{bmatrix} 2 & 0 \\ 0 & 50 \end{bmatrix}, \ \kappa = 25$$

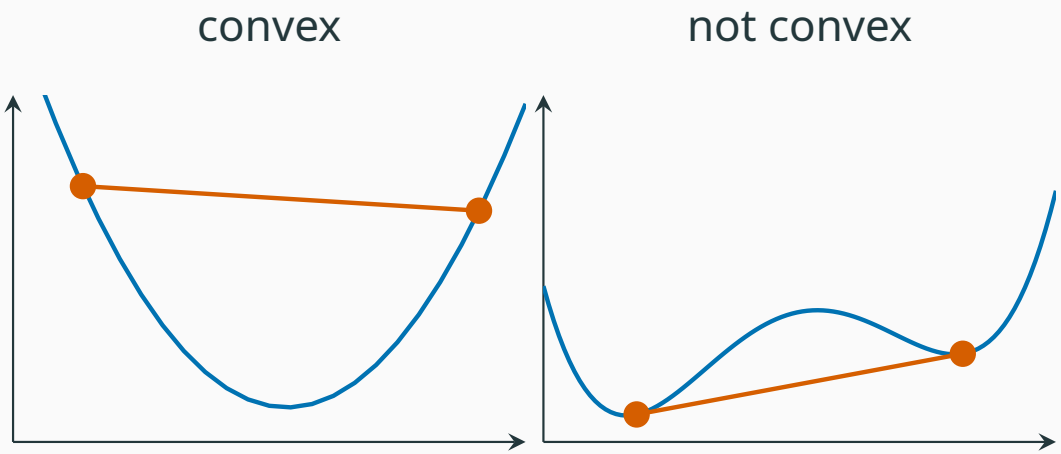
- $f(x, y) = x^2 + 25y^2$ has this Hessian: a bowl 25 times more curved along y than along x

Gradient and Hessian

$$\nabla f(x) = \begin{bmatrix} \dfrac{\partial f}{\partial x_1} \\ \vdots \\ \dfrac{\partial f}{\partial x_n} \end{bmatrix} \qquad H(x) = \begin{bmatrix} \dfrac{\partial^2 f}{\partial x_1^2} & \cdots & \dfrac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \dfrac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

- ∇f: n numbers, the direction of **steepest ascent**; $-\nabla f$ points downhill
- H: n^2 numbers, the **curvature** in every pair of directions
- Eigenvalues of H: all > 0 minimum, all < 0 maximum, mixed signs **saddle**

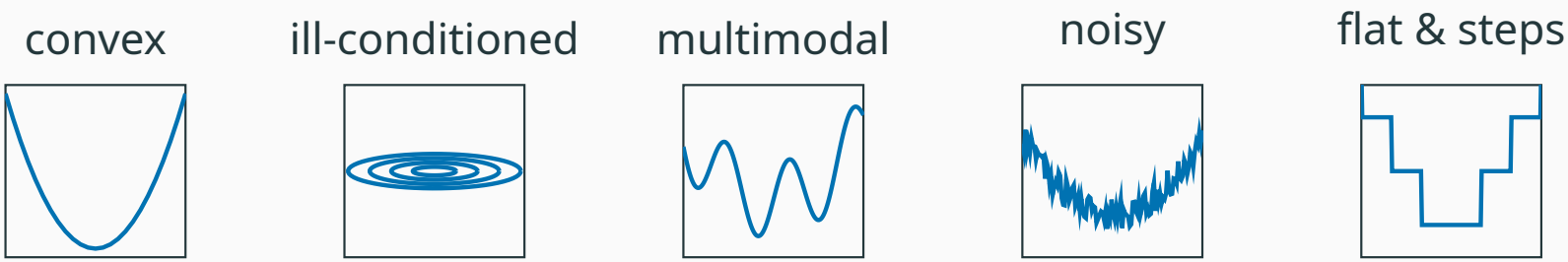
Convexity



$$f(tx+(1-t)y) \leq tf(x)+(1-t)f(y) \qquad \forall x, y,\ t \in [0,1]$$

- The chord between any two points lies **above** the curve
- Convex $\Rightarrow$ every local minimum is **global**: nothing can trap the search
- Linear and logistic regression are convex; neural networks are not

Common Landscapes



- **Convex:** one basin; any downhill method reaches the minimum
- **Ill-conditioned** (2D contours): steep in one direction, flat in the other
- **Multimodal:** many basins; the start decides the result
- **Noisy:** the same point returns a different value each time
- **Flat or discontinuous:** the gradient is zero or undefined
- Also common, often mixed: curved valleys (Rosenbrock), ridges, deceptive basins

Optimization in Machine Learning

Training Is Optimization

$$\theta^\star = \arg\min_\theta \; \frac{1}{N} \sum_{i=1}^{N} \ell\big(f_\theta(x_i),\, y_i\big) \;+\; \lambda\, \Omega(\theta)$$

- ℓ: the loss on **one** example; the average is the **empirical risk**
- $\lambda\, \Omega(\theta)$: an optional **regularization** term that penalizes complex models (the $L(H)$ part of MDL); $\lambda = 0$ removes it (Topic 3)
- Minimizing the training loss is only a *means*: the goal is low **test** error (Class 01)
- So we rarely need the exact $\theta^\star$; a good θ found cheaply is often enough

Common Losses

Loss	$\ell(\hat{y}, y)$	Used for	Smooth?
Squared error	$(\hat{y} - y)^2$	regression	yes
Huber	$r^2/2$ near 0, linear beyond δ	robust regression	yes
Cross-entropy	$-\log \hat{p}_y$ (prob. of the true class)	classification	yes
0/1 loss	$[\hat{y} \neq y]$	what we report	**no**

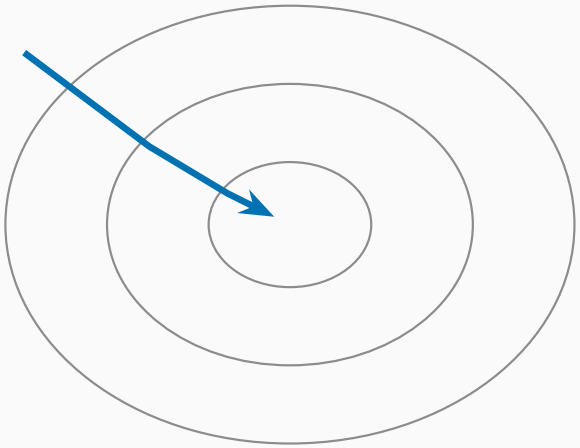
- The loss we *care* about (errors) is often not the loss we can *optimize* (smooth surrogates)
- A smooth loss has a useful gradient; a step-shaped loss does not

Parameters and Hyperparameters

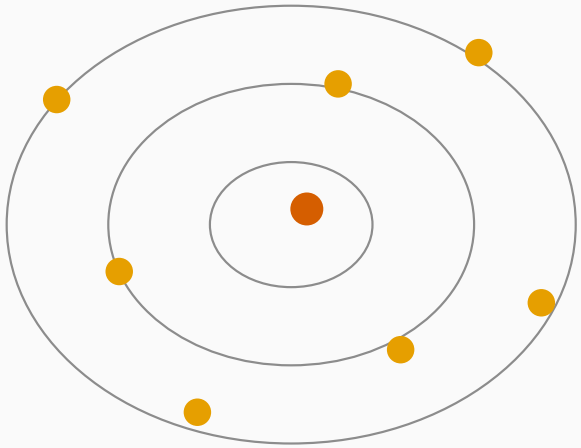
	Parameters	Hyperparameters
Examples	weights, biases, centroids	learning rate, depth, k, λ
How many	thousands to billions	a handful
Gradient available?	usually yes	usually **no**
One evaluation costs	microseconds	a full training run
Typical optimizer	gradient-based	blind (search)

- Both are optimization problems; they sit at different levels
- Blind methods also fit models directly when the loss has no gradient

Two Families of Optimizers

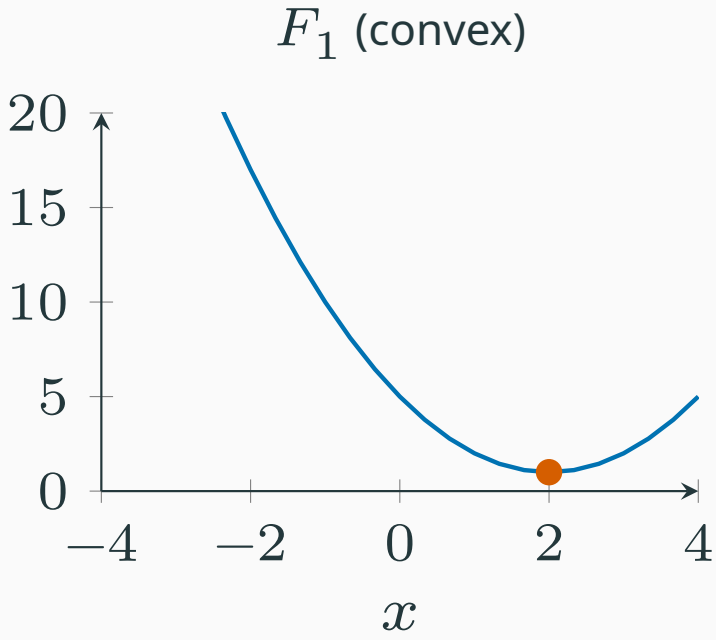


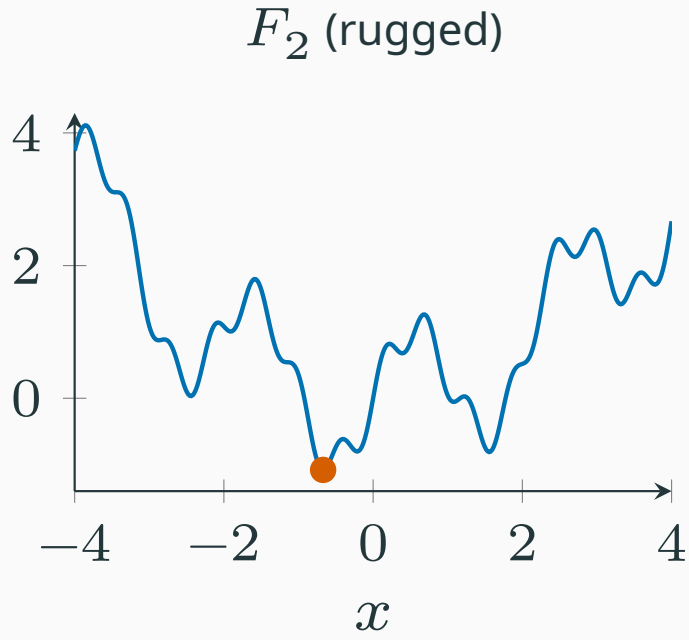
Gradient-based
one point, follows $-\nabla f$
needs derivatives, very efficient



Blind (derivative-free)
only values $f(x)$,
often a population
works on any black box,
needs more evaluations

Two Test Functions

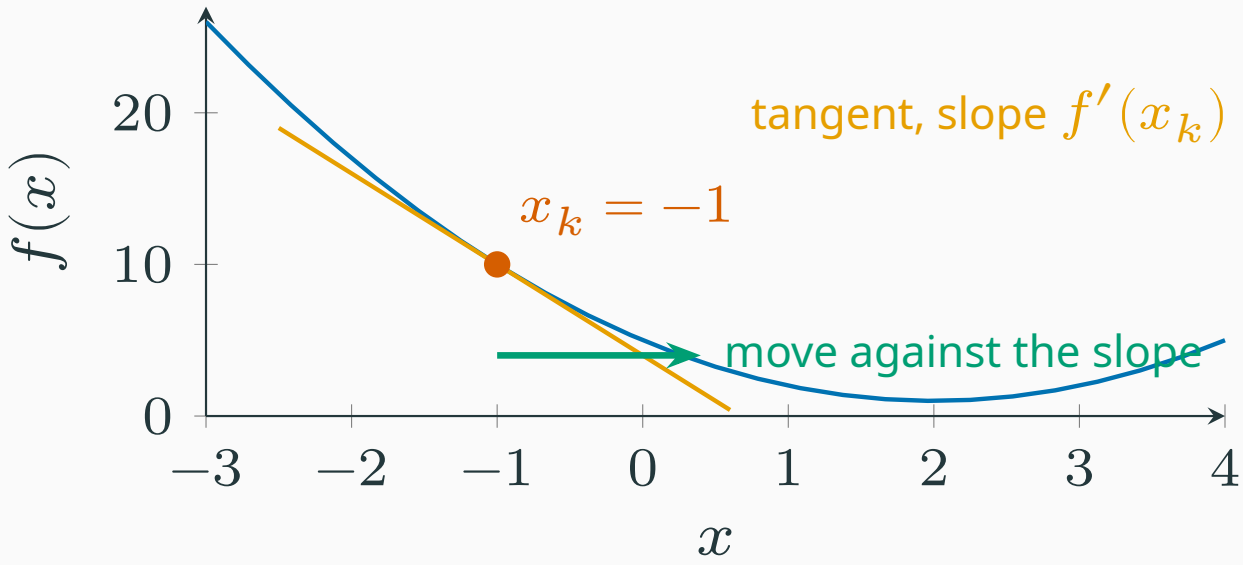




- $F_1(x) = (x-2)^2 + 1$: convex, minimum at $x^\star = 2$
- $F_2(x) = x^2/5 + \sin 3x + 0.3 \sin 11x$: a smooth bowl (the signal) plus fast ripples that look like **noise**; global minimum $x^\star \approx -0.67$, surrounded by local minima
- The **same two functions** are used by both demo notebooks (`f_convex`, `f_rugged`), so the two families can be compared

Gradient-Based Optimization

The Derivative Is a Local Slope



- $f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$: how much f changes per unit step in x
- Negative slope: f decreases to the **right**, so step right; positive slope: step left
- In n-D, $-\nabla f$ is the direction of **steepest descent**

Gradient Descent

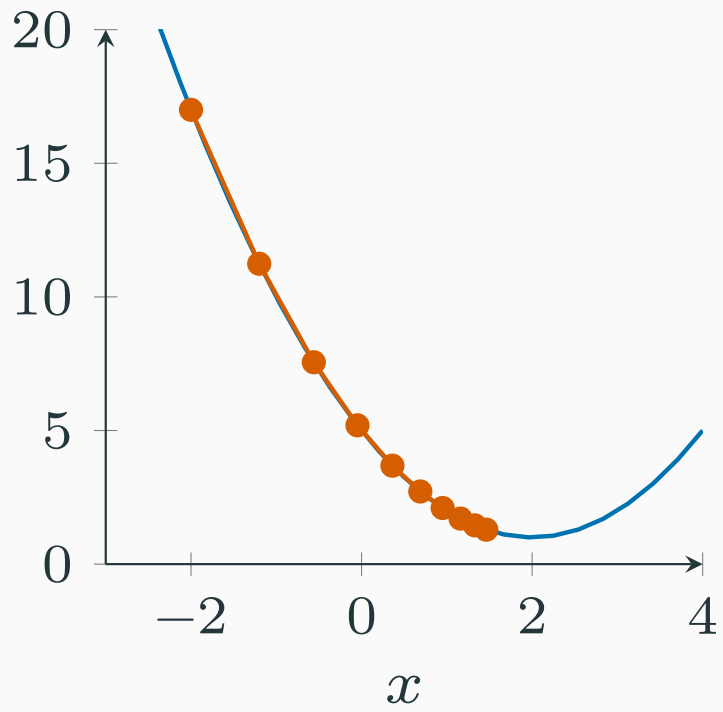
$$x_{k+1} = x_k - \eta \nabla f(x_k)$$

- $\eta > 0$: the **learning rate** (step size)
- It follows from the first-order Taylor model $f(x + \delta) \approx f(x) + \nabla f(x)^\top \delta$: for a small step of fixed length, $\delta \propto -\nabla f$ decreases f the most
- The step is **proportional to the slope**: large far away, small near the minimum
- Stop when $\|\nabla f\|$ is tiny, the loss stops improving, or the budget runs out

Worked Example: GD by Hand

$$f(x) = (x-2)^2 + 1, \quad f'(x) = 2(x-2)$$
$$x_0 = -2, \ \eta = 0.1$$

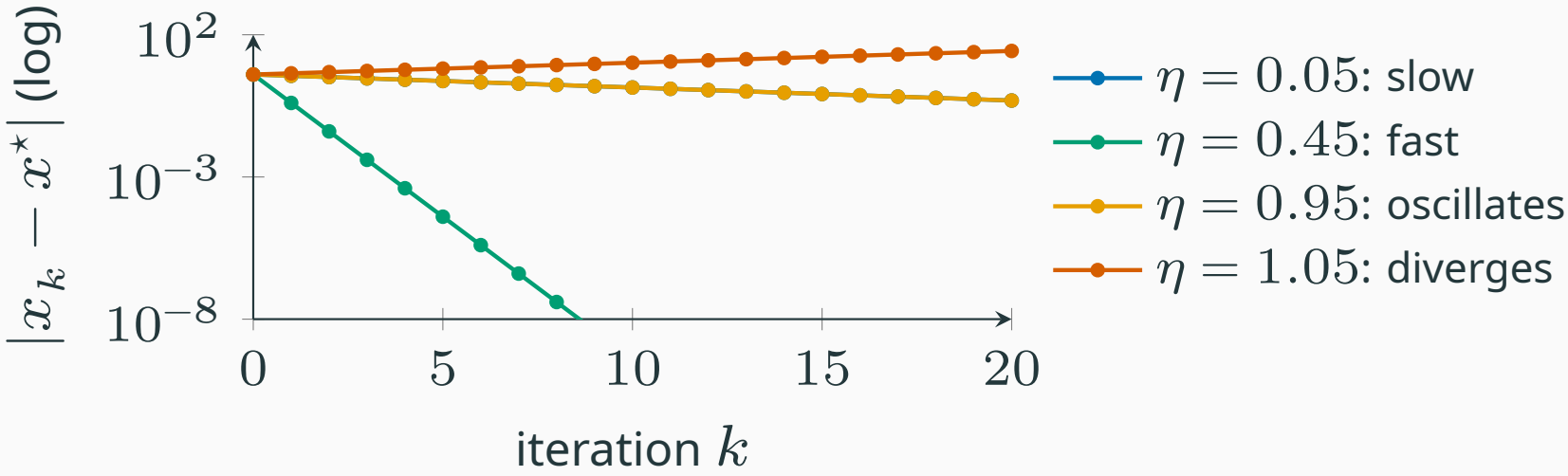
k	x_k	$f(x_k)$	$f'(x_k)$
0	-2.000	17.000	-8.000
1	-1.200	11.240	-6.400
2	-0.560	7.554	-5.120
3	-0.048	5.194	-4.096
4	0.362	3.684	-3.277
5	0.689	2.718	-2.621



- $x_1 = -2 - 0.1 \cdot (-8) = -1.2$; the steps shrink as the slope flattens

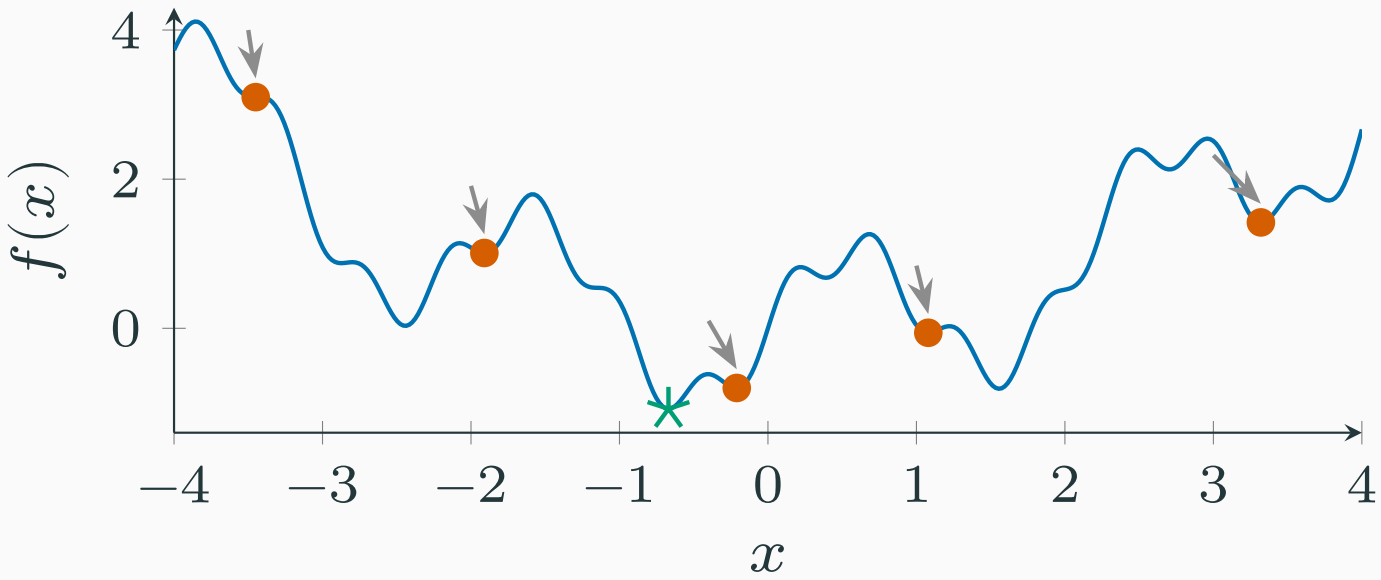
The Learning Rate

$$x_{k+1} - 2 = (1 - 2\eta)(x_k - 2) \;\Rightarrow\; |x_k - x^\star| = |1 - 2\eta|^k |x_0 - x^\star|$$



- Too small: slow. Too large: overshoots and oscillates (same error size as $\eta = 0.05$, but the sign flips), and finally **diverges**
- The safe range depends on the **curvature**: here $f'' = 2$ and GD is stable for $\eta < 2/f'' = 1$

A Rugged Function: Local Minima



$$f'_{rugged}(x) = \frac{2x}{5} + 3\cos 3x + 3.3\cos 11x$$

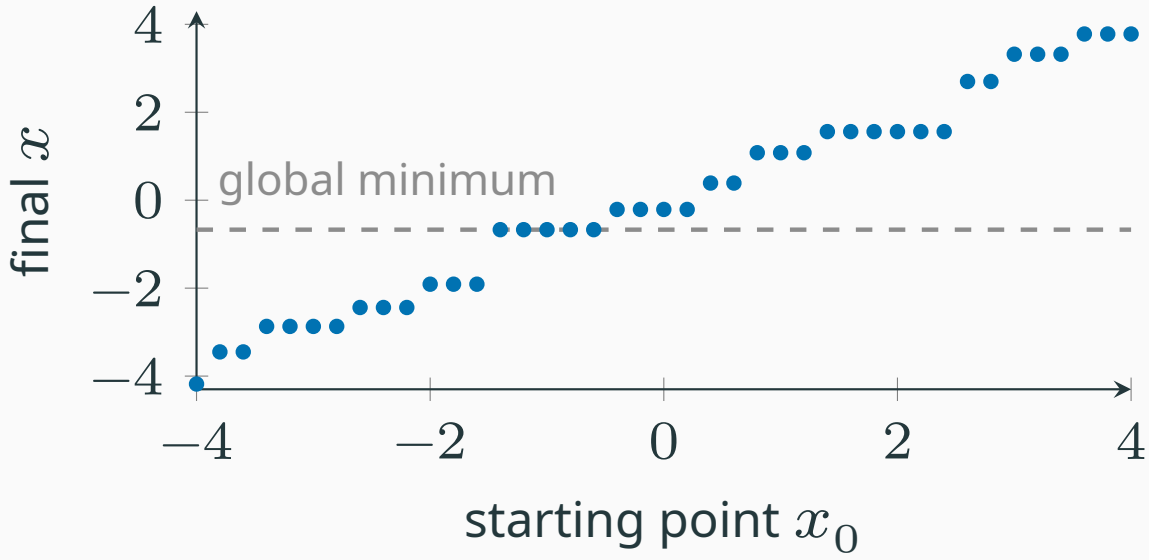
- From 5 starting points, GD stops in the **nearest valley** (red); none reaches the global minimum (green)
- Over 400 starts on $[-4, 4]$, only **15%** end in the global basin: GD is a **local** method

Where Does the Gradient Come From?

Method	How	Accuracy	Cost for n parameters
Symbolic	derive by hand (or CAS)	exact	human time; error-prone
Numerical	$\frac{f(x+h)-f(x-h)}{2h}$	$\approx$ 6–8 digits	$2n$ evaluations of f
Automatic	chain rule on the program	exact (machine precision)	$\approx$ 2–4 $\times$ one evaluation

- **Reverse-mode automatic differentiation** (backpropagation) gets *all* n partial derivatives for the price of a few evaluations of f
- This is why models with millions of parameters are trainable at all
- In this course: **JAX** (`jax.grad`), also the backend of Keras 3

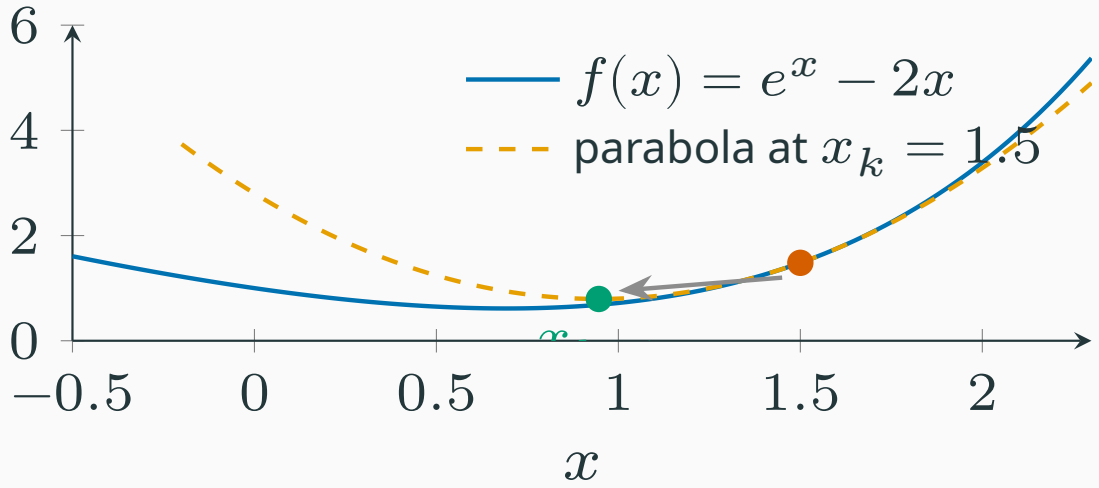
Live Demo G1–G3: Gradient Descent



- Notebook `01_gradient_based.ipynb`: **G1** derivative by hand, by finite differences and with `jax.grad`; **G2** four learning rates; **G3** basins of attraction
- GD from 41 starts ($\eta = 0.02$, 300 steps). Each flat step is one **basin of attraction**: the start decides the answer

Newton’s Method

Use the Curvature Too



$$f(x_k + \delta) \approx f(x_k) + f'(x_k)\,\delta + \tfrac{1}{2} f''(x_k)\,\delta^2$$

$$\Longrightarrow \quad x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}$$

- Fit a **parabola** (second-order Taylor) at x_k and jump to its minimum
- No learning rate: the step length comes from the curvature

Newton in n Dimensions

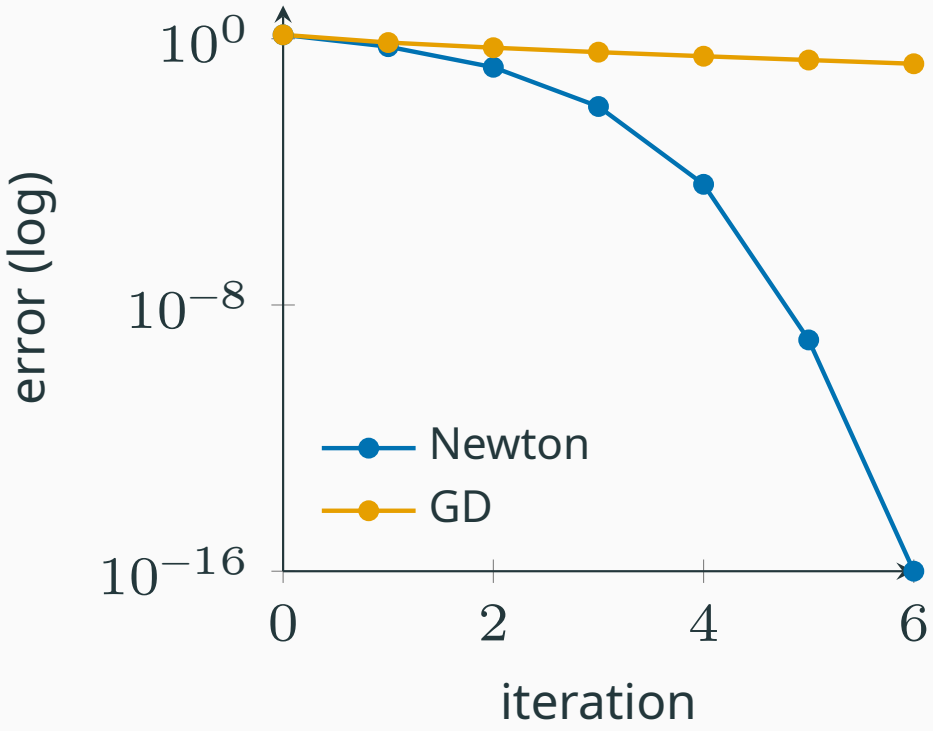
$$\theta_{k+1} = \theta_k - H(\theta_k)^{-1} \nabla \mathcal{L}(\theta_k)$$

- H: the $n \times n$ Hessian. In practice solve $H\,\delta = -\nabla \mathcal{L}$ (never invert H)
- On a **quadratic** loss (e.g. linear regression with MSE) the Taylor model is exact: **one step** reaches the minimum
- Each step: n^2 second derivatives and an $O(n^3)$ solve
- **Damped** Newton: $(H + \lambda I)^{-1}$ blends Newton ($\lambda \to 0$) with a small gradient step (λ large)
- **Quasi-Newton** (BFGS, L-BFGS): build an approximation of H^{-1} from successive gradients

Worked Example: Quadratic Convergence

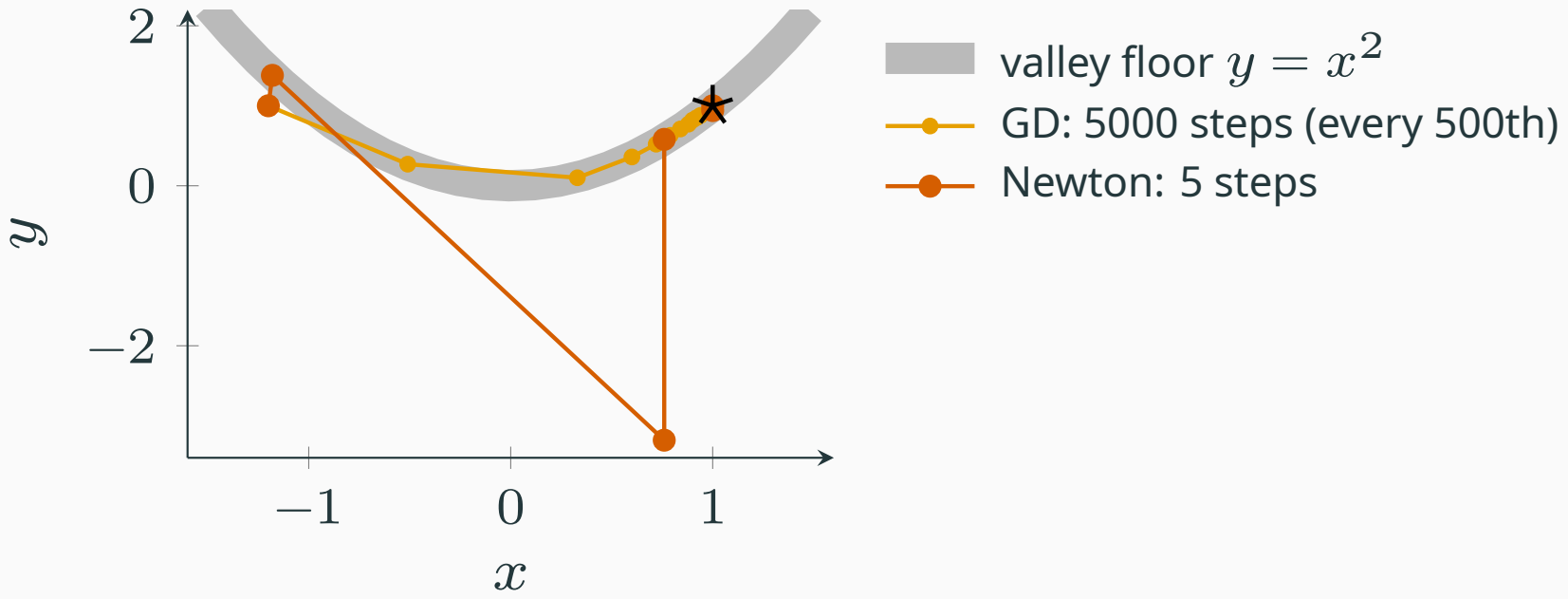
$$f(x) = e^x - 2x, \qquad x^\star = \ln 2, \;\; x_0 = 2$$

k	Newton $\lvert x_k - x^\star \rvert$	GD ($\eta = 0.1$)
0	1.3	1.3
1	$5.8 \cdot 10^{-1}$	$7.7 \cdot 10^{-1}$
2	$1.4 \cdot 10^{-1}$	$5.4 \cdot 10^{-1}$
3	$9.2 \cdot 10^{-3}$	$4.0 \cdot 10^{-1}$
4	$4.2 \cdot 10^{-5}$	$3.0 \cdot 10^{-1}$
5	$8.9 \cdot 10^{-10}$	$2.3 \cdot 10^{-1}$
6	0 (machine)	$1.8 \cdot 10^{-1}$



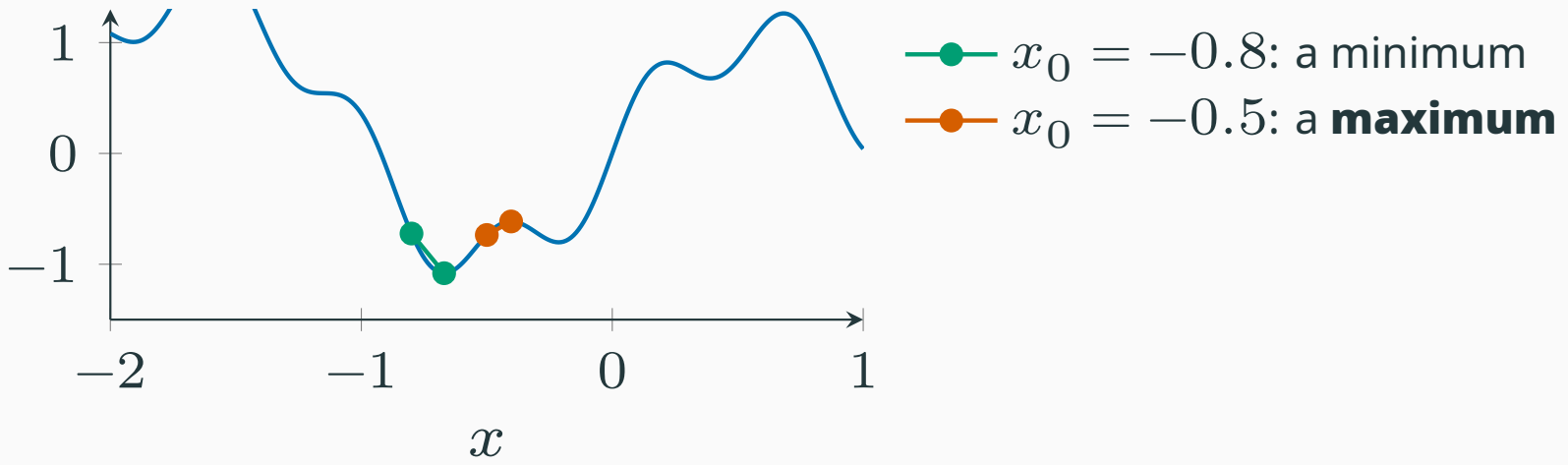
- Near the minimum Newton **doubles the correct digits** at every step: $9 \cdot 10^{-3} \to 4 \cdot 10^{-5} \to 9 \cdot 10^{-10}$

Newton on Rosenbrock



- $f(x, y) = (1 - x)^2 + 100\,(y - x^2)^2$, minimum at $(1, 1)$, start at $(-1.2, 1)$
- Newton: exact after **5 steps** (with a wild jump on the way). GD: still $f \approx 4 \cdot 10^{-3}$ after 5000

When Newton Fails



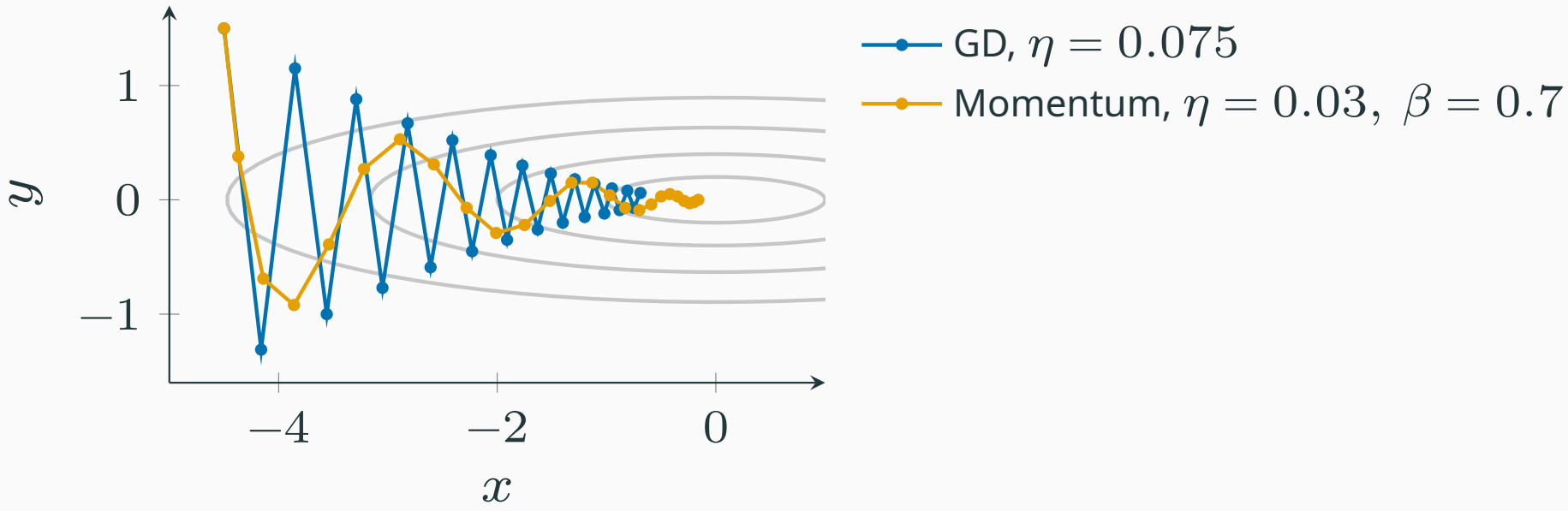
- Newton solves $f'(x) = 0$: it cannot tell minima, maxima and saddles apart
- Where $f'' < 0$ the parabola opens downwards and the step goes **uphill**
- Where $f'' \approx 0$ the step explodes: from $x_0 = -0.9$ it ends at $x = 7.28$
- Cost: $O(n^2)$ memory and $O(n^3)$ time per step, impossible for millions of weights

Live Demo G4: Newton's Method

- Notebook `01_gradient_based.ipynb`, section **G4**
- One step on a quadratic; quadratic convergence on $e^x - 2x$
- Minimum, maximum and explosion on the rugged function
- `jax.hessian` and `jnp.linalg.solve` on Rosenbrock
- Question: which of these would you use to train a network with 10^6 weights?

Momentum and Adaptive Methods

Ill-Conditioning: the Zig-Zag



- $q(x, y) = \frac{1}{2}(x^2 + 25y^2)$: 25 times steeper in y than in x (first 25 steps)
- GD's step is capped by the **steep** direction ($\eta < 2/25$), so it crawls along the **flat** one
- Momentum averages out the zig-zag and accumulates speed along the valley

Momentum: a Heavy Ball



$$v_{k+1} = \beta\, v_k + \nabla \mathcal{L}(\theta_k), \qquad \theta_{k+1} = \theta_k - \eta\, v_{k+1}$$

- v: an exponentially weighted **sum of past gradients**; $\beta \approx 0.9$ is the "friction"
- Consistent directions add up; oscillating directions cancel
- **Nesterov:** evaluate the gradient at the look-ahead point $\theta_k - \eta\beta v_k$ (corrects earlier)

Adaptive Step Sizes: RMSProp

$$s_{k+1} = \rho\, s_k + (1-\rho)\, g_k^2, \qquad \theta_{k+1} = \theta_k - \eta\, \frac{g_k}{\sqrt{s_{k+1}} + \epsilon}$$

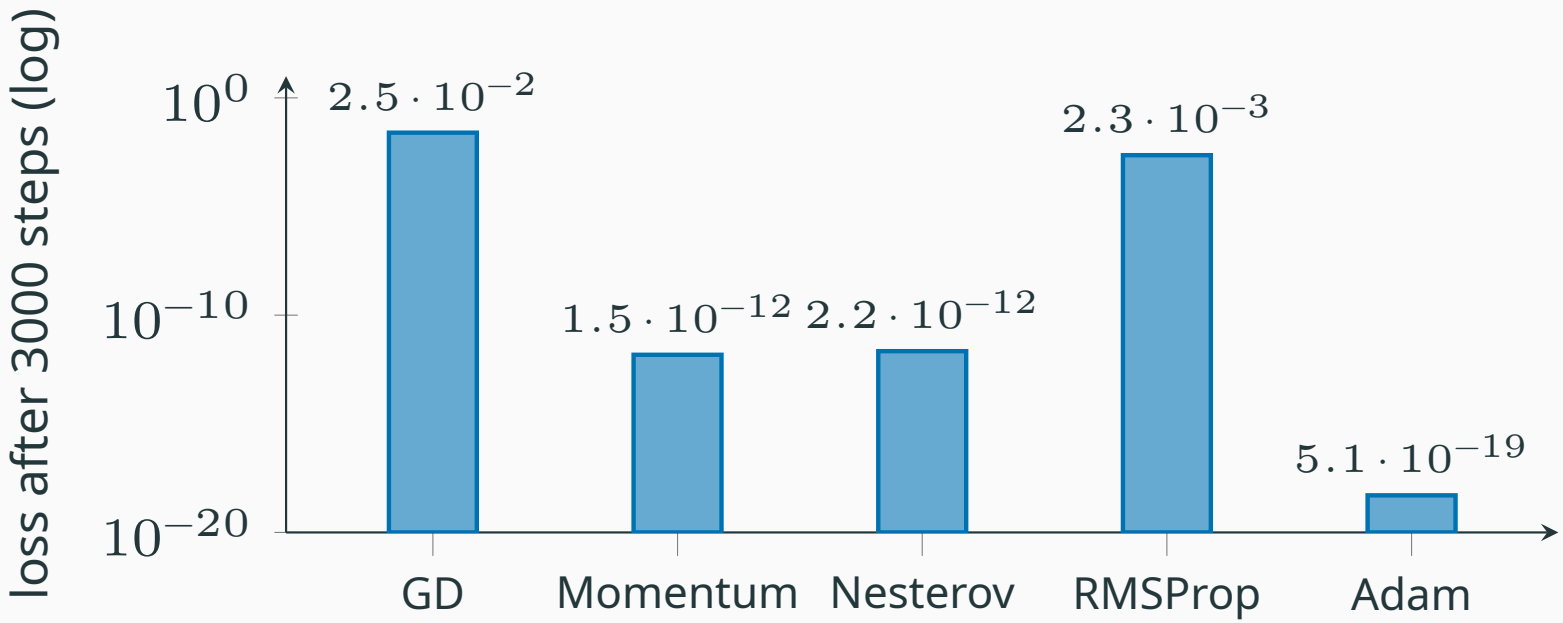
- $g_k = \nabla \mathcal{L}(\theta_k)$; squares and divisions are **per coordinate**
- s tracks the recent **size** of each gradient coordinate
- Steep coordinates get smaller steps, flat ones larger: every direction moves at a similar pace
- A cheap, **diagonal** stand-in for Newton's curvature scaling (n numbers instead of n^2)
- History: AdaGrad (2011) sums all past g^2; RMSProp (Hinton, 2012) forgets old ones

Adam = Momentum + RMSProp

$$m_{k+1} = \beta_1 m_k + (1-\beta_1)\, g_k, \quad s_{k+1} = \beta_2 s_k + (1-\beta_2)\, g_k^2$$
$$\hat{m} = m_{k+1}/(1-\beta_1^{k+1}), \qquad \hat{s} = s_{k+1}/(1-\beta_2^{k+1})$$
$$\theta_{k+1} = \theta_k - \eta\, \hat{m}/(\sqrt{\hat{s}} + \epsilon)$$

- m: the **direction** (momentum); s: the **scale** (RMSProp)
- Defaults: $\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ (Kingma and Ba, 2015)
- Bias correction: m and s start at 0, so early averages are too small without it
- The default optimizer of deep learning: `keras.optimizers.Adam`

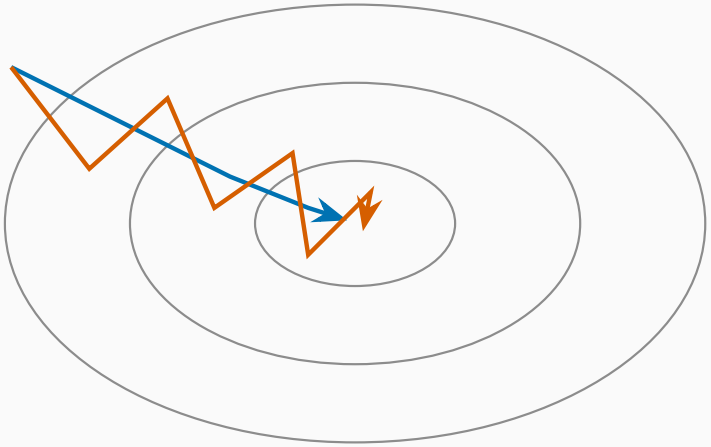
Comparing the Optimizers



- Rosenbrock from $(-1.2, 1)$, learning rates tuned roughly by hand: 10^{-3} (GD, momentum), $3 \cdot 10^{-3}$ (RMSProp), 0.05 (Adam)
- With a constant η, RMSProp and Adam **hover** around the minimum with steps of order η: in practice η is decayed

Stochastic Gradient Descent

$$\nabla \mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \nabla \ell_i(\theta) \approx \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla \ell_i(\theta)$$



full batch: smooth, expensive

mini-batch: noisy, cheap

- A **mini-batch** $\mathcal{B}$ of $B \ll N$ examples gives a cheap, unbiased, **noisy** estimate of the gradient
- One pass over all data = one **epoch**; $B = 32$–256 is typical
- The noise is useful: it helps escape sharp minima and saddle points

Live Demo G5: Momentum, RMSProp, Adam, SGD

- Notebook `01_gradient_based.ipynb`, section **G5**
- All four optimizers written in a few lines each, on the ill-conditioned bowl and on Rosenbrock
- Momentum on the rugged 1D function: from $x_0 = -3.7$ it rolls into the global basin where GD stops early; over 400 starts, the mean final loss drops from 0.38 to 0.05
- SGD with batch sizes 1000, 16 and 1 on a line fit: the smaller the batch, the noisier the path near the minimum

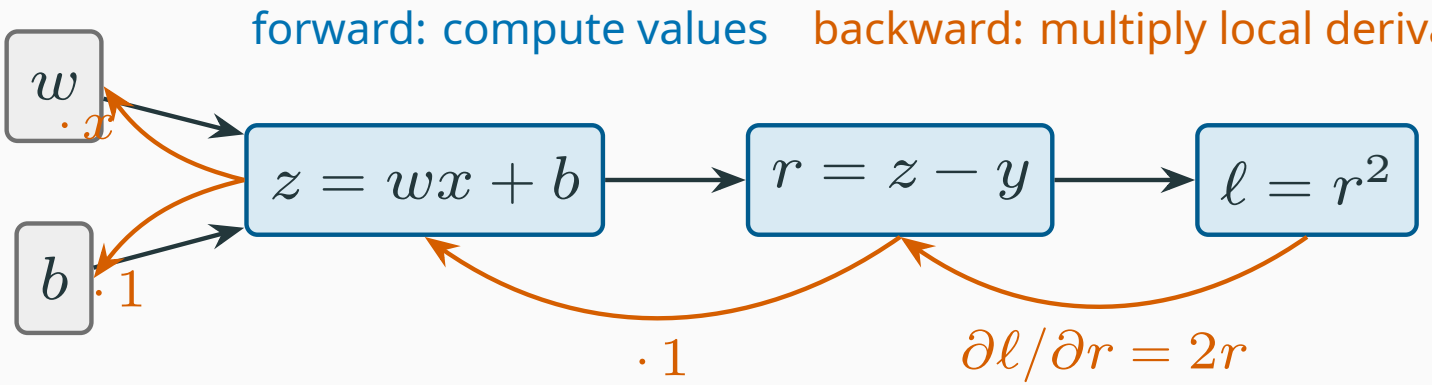
Automatic Differentiation with JAX

JAX: NumPy That Can Differentiate

Transform	Returns
`jax.grad(f)`	a function computing ∇f (reverse mode)
`jax.value_and_grad(f)`	f and ∇f in one pass
`jax.hessian(f)`	the matrix of second derivatives
`jax.jit(f)`	f compiled with XLA (CPU or GPU)
`jax.vmap(f)`	f vectorized over a batch axis

- Write the model and the loss with `jax.numpy` (same API as NumPy); JAX derives the rest
- Transforms **compose**: `jax.jit(jax.vmap(jax.grad(loss)))`
- Keras 3 on the JAX backend uses exactly this machinery to train networks

The Chain Rule on a Graph



- **Forward pass:** evaluate the program and remember intermediate values
- **Backward pass:** apply the chain rule from the output back to every input: $\frac{\partial \ell}{\partial w} = 2r \cdot 1 \cdot x$
- One backward pass gives **all** partial derivatives: this is backpropagation (Topic 4)

JAX in Five Lines

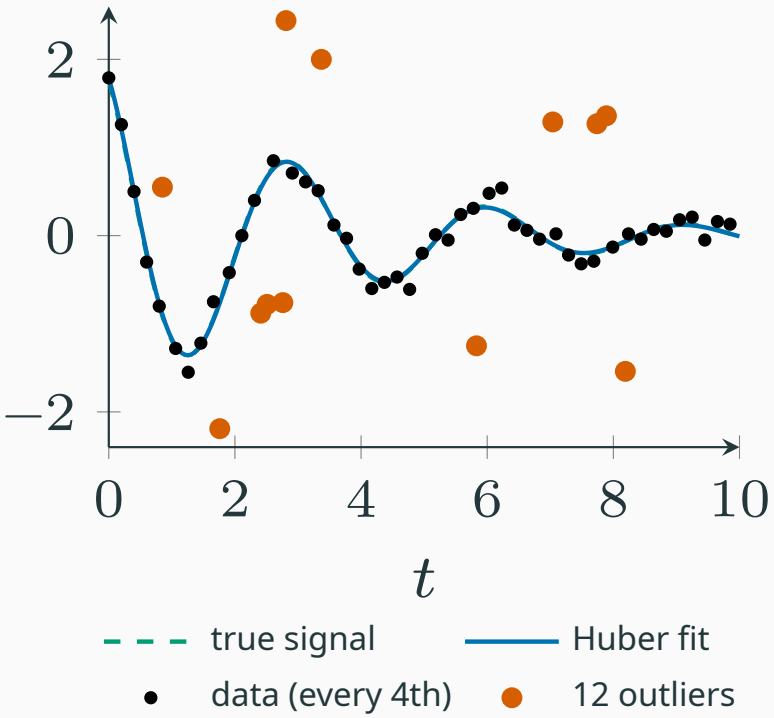
```python
import jax, jax.numpy as jnp

def loss(theta, x, y):                          # any model, any loss
    return jnp.mean((theta[0] * x + theta[1] - y) ** 2)

grad = jax.jit(jax.grad(loss))                   # derived and compiled
for _ in range(100):
    theta = theta - 0.1 * grad(theta, x, y) # gradient descent
```

- No derivative written by hand; changing the model or the loss changes nothing else
- `jax.jit`: the gradient of Rosenbrock drops from about **8 ms** to about **7** μ**s** per call, $\approx 1000\times$ faster (the first call compiles)

A Custom Model with a Custom Loss

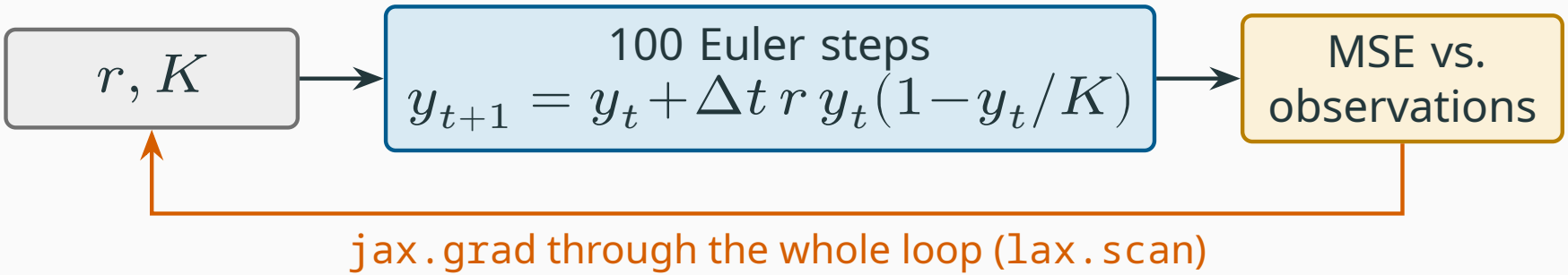


$$\hat{y}(t) = A\, e^{-\lambda t} \cos(\omega t + \varphi)$$

	A	λ	ω	φ
true	2.000	0.300	2.000	0.500
MSE	2.023	0.326	1.963	0.458
Huber	2.017	0.307	2.002	0.483

- 200 noisy points and 12 outliers; **Huber loss**: quadratic for small residuals, linear for large ones
- Same model, two losses, zero derivatives by hand: Huber is pulled less by the outliers

Differentiating Through a Simulation



- Some models have no formula, only a **numerical integration** (an ODE solver, a physics simulation)
- JAX differentiates the solver itself: from $(r, K) = (0.5, 3)$, Adam recovers $r = 0.805$ (true 0.8) and $K = 4.958$ (true 5.0)
- Use this whenever a model or a loss is not in a library

Live Demo G6: JAX for Custom Models

- Notebook `01_gradient_based.ipynb`, section **G6**
- `jit` timing; the damped oscillator with MSE and Huber; logistic growth fitted through its ODE solver
- Exercise: change the Huber δ, or the model, and rerun: nothing else changes

Blind Optimization

When There Is No Useful Gradient

black box
simulator, lab experiment, game

no derivative
0/1 loss, ranks, discrete choices

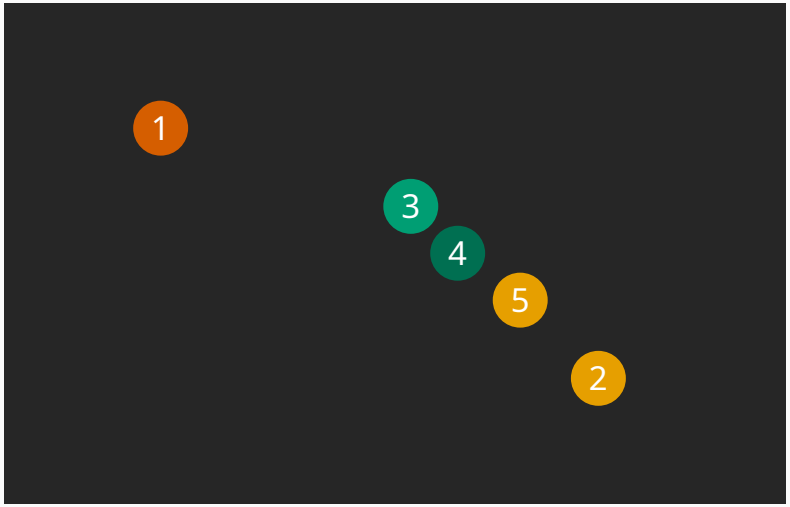
noisy
each evaluation differs

hyper-parameters
one value = one training run

multimodal
gradients find only the nearest valley

- A **blind** (derivative-free, zeroth-order) optimizer only asks: *what is* $f(x)$ *here?*
- It needs no formula, tolerates discontinuities and noise, and can search globally
- The price: many more evaluations, and weaker guarantees

Activity: Be the Optimizer



Hidden 2D function
5 evaluations per problem
no landscape, no $f^\star$, no gradient
only the numbers you asked for

- **blindgame**: minimize hidden functions by hand, with a class leaderboard (link given in class)
- Inspired by the **GECCO 2025 Fun Competition**: *manual* optimization by human experts
- Afterwards, the algorithms of this lecture get the same budget: who does better, you or them?
- Think about it while you play: where do you place the first point? When do you stop exploring?

Exploration versus Exploitation



- **Exploration:** sample new regions; finds the right basin, but is imprecise
- **Exploitation:** refine around the best point; precise, but can be trapped
- Every blind algorithm is a **schedule** between the two: first explore, then exploit

Random Search and Hill Climbing

Random search

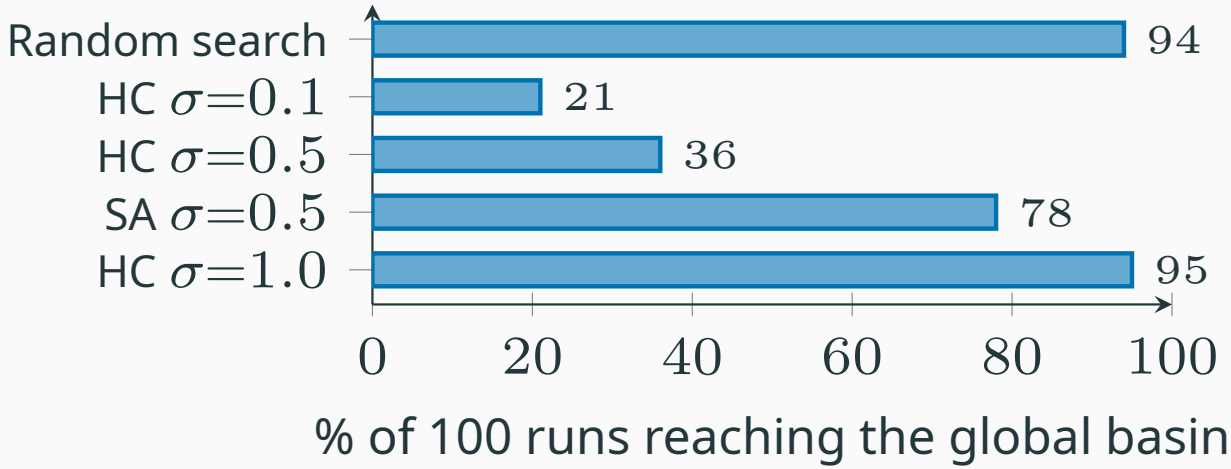
1. sample $x \sim \mathcal{U}(\text{box})$
2. keep the best seen so far

No memory of *where* good points are.

Hill climbing

1. $x' = x + \mathcal{N}(0, \sigma^2)$
2. if $f(x') < f(x)$: $x \leftarrow x'$

A local search: the step size σ sets how far it looks.



- f_{rugged}, 200 evaluations, random start. In **1D** random search is excellent; in 10D it will be the worst

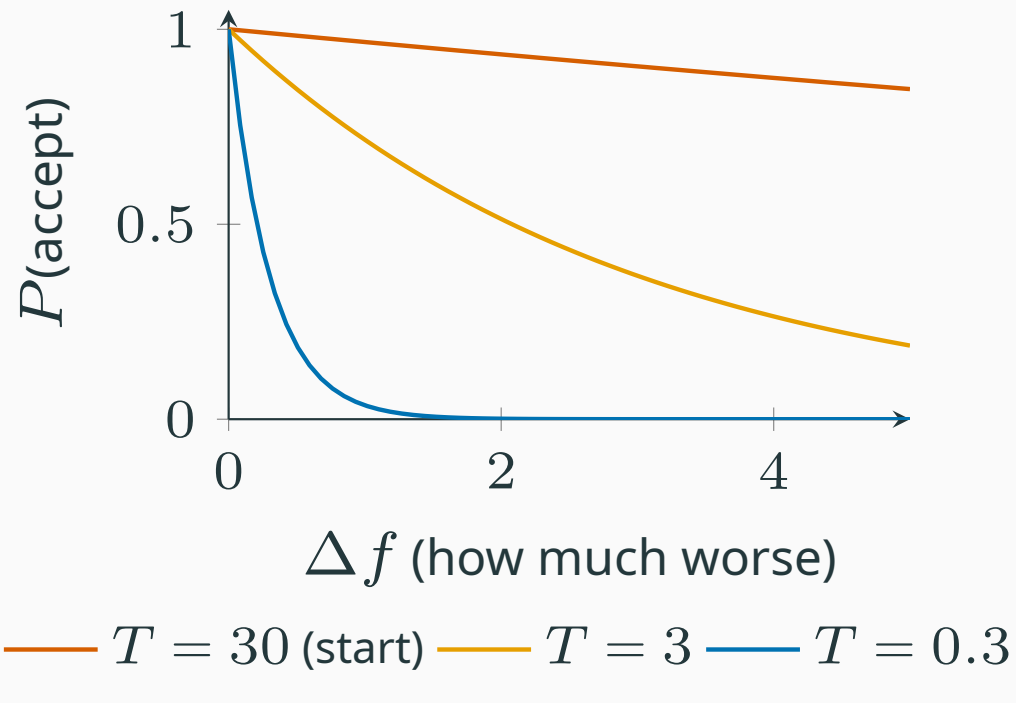
Simulated Annealing

Accept a **worse** point ($\Delta f > 0$) with probability

$$P(\text{accept}) = e^{-\Delta f / T}$$

Cooling schedule (pyBlindOpt):

$$T_t = \frac{T_0}{t+1}$$



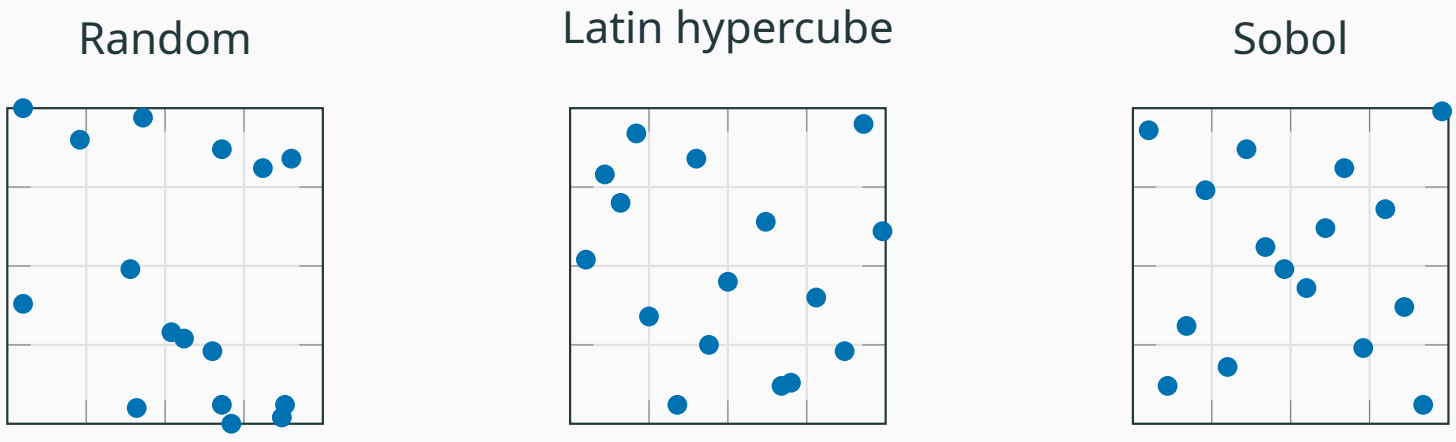
- Hot: accepts almost anything (**explores**); cold: only improvements (**exploits**, like hill climbing)
- Inspired by annealing in metallurgy: slow cooling lets the atoms find a low-energy state
- On f_{rugged}: 78% of runs reach the global basin against 36% for hill climbing with the same σ

Live Demo B1: Local Search

- Notebook `02_blind_optimization.ipynb`, section **B1**
- Random search, hill climbing and simulated annealing from $x_0 = 3$, 200 evaluations each
- Success rates over 100 random starts, for several step sizes
- On f_{convex} all three work, but each needs **200 evaluations** where GD needed about 25 gradient steps

Initialization

Where Do We Start?



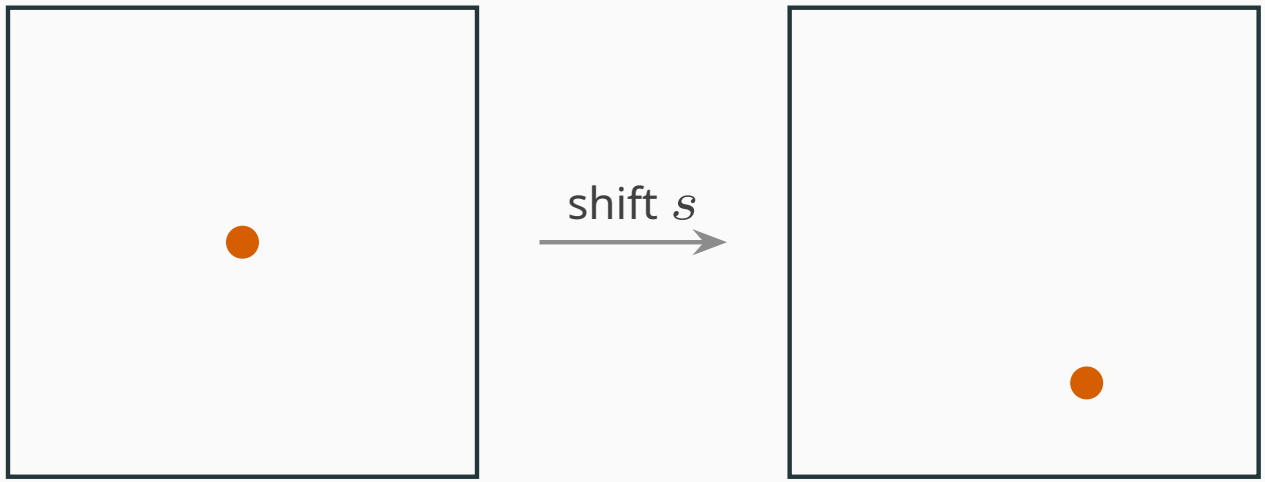
- 16 points from pyBlindOpt's samplers (`RandomSampler`, `HLCSampler`, `SobolSampler`)
- **Random:** clumps and holes. **Latin hypercube:** one point per row strip and per column strip
- **Sobol:** a low-discrepancy sequence, spread evenly at every scale. **Chaotic:** a logistic map

Opposition-Based Strategies



- **OBL:** evaluate every sample and its opposite, keep the best N of the $2N$
- **QOBL:** the opposite is drawn between the centre and $\breve{x}$
- **OBLESA:** OBL plus probes placed in the **empty space** no sample covered (EmptySpaceSearch), then the best N with a diversity bonus
- Samplers are free; strategies **spend evaluations** ($2N$, $3N$) before the search starts

A Benchmark Pitfall: the Centre

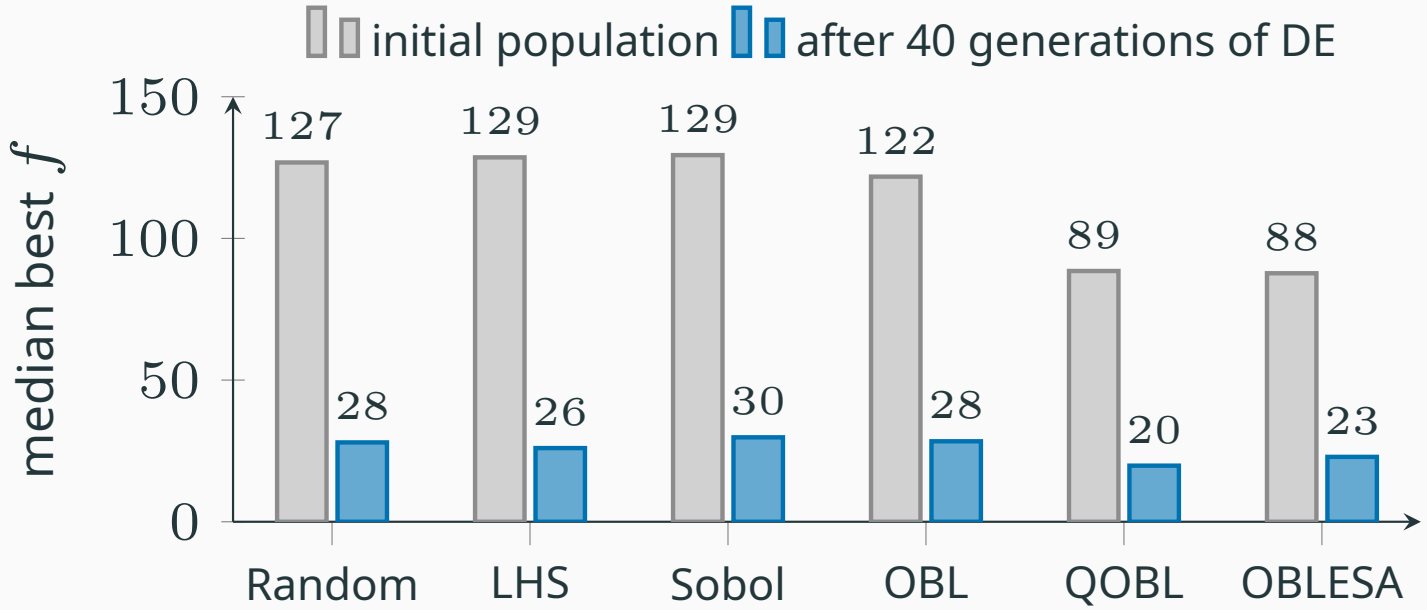


textbook: $x^\star$ at the centre

realistic: $x^\star$ anywhere

- Sphere, Rastrigin, Ackley have $x^\star = 0$ at the centre of a symmetric box $[-5, 5]^d$
- There **OBL is useless** ($f(-x) = f(x)$: the opposite has the same value) and **QOBL looks brilliant** (it samples towards the centre)
- Always test on **shifted** functions $f(x - s)$, as the BBOB benchmarks do

Does Initialization Matter?



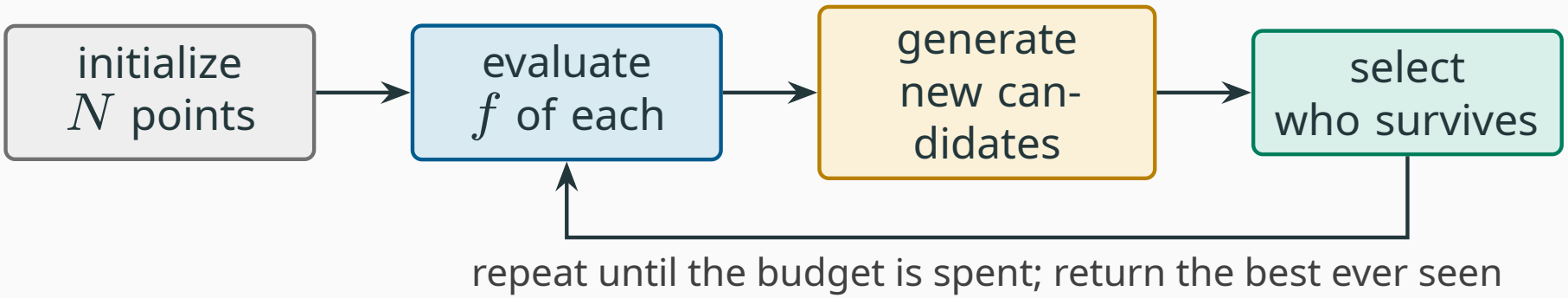
- Shifted Rastrigin, $d = 10$, 20 individuals, **30 seeds** per method (lower is better)
- Better spreading (LHS, Sobol) barely matters with 20 points in 10D; **selecting** the start (QOBL, OBLESA) gives DE a head start that survives
- The effect depends on the algorithm: EGWO hardly responds to a better start

Live Demo B2: Initialization

- Notebook `02_blind_optimization.ipynb`, section **B2**
- The four samplers in 2D, with the closest pair of points as a crude coverage measure
- OBL, QOBL and OBLESA on a shifted Rastrigin landscape
- 180 DE runs: initial and final best for each initializer, as box plots

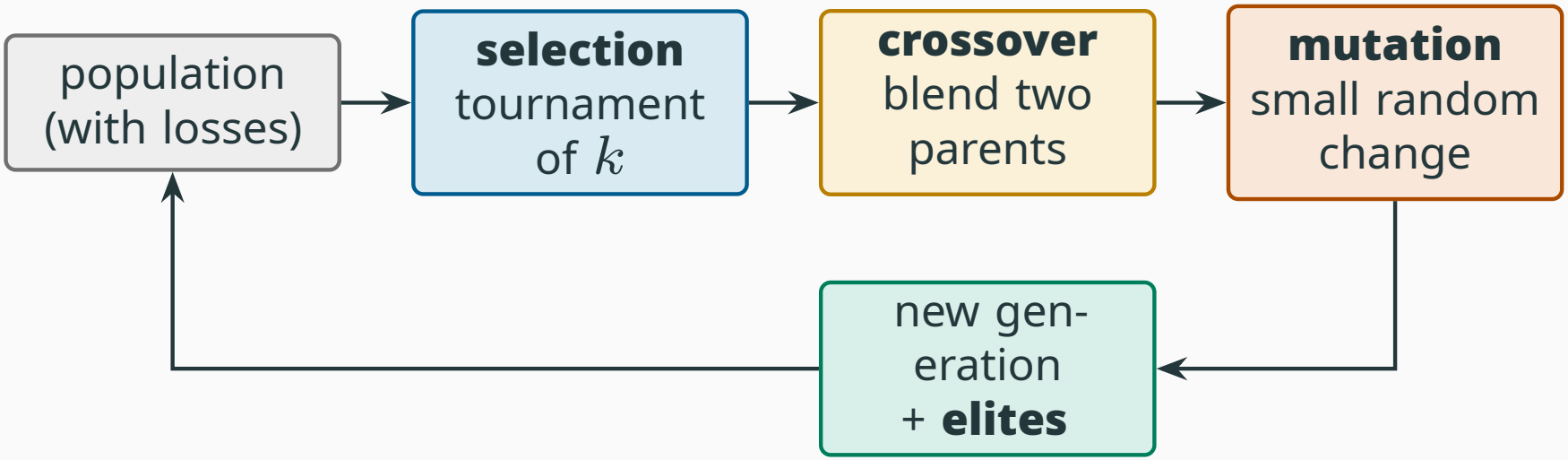
Population-Based Algorithms

One Loop, Many Algorithms



- The algorithms differ only in **generate** and **select**
- pyBlindOpt implements this loop once (`Optimizer.optimize`); each algorithm supplies its operators
- Budget = $N \times (\text{epochs} + 1)$ evaluations: compare algorithms at **equal budget**

Genetic Algorithm



- **Selection** (who reproduces) sets the pressure; **crossover** mixes good solutions
- **Mutation** reaches values no parent has; **elitism** copies the best unchanged
- Real-coded GA: an individual is simply the vector $\theta \in \mathbb{R}^d$

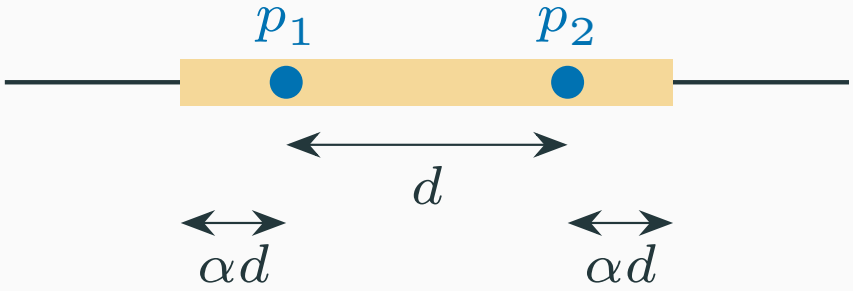
GA Operators

Tournament (k): pick k at random, keep the best

- $k = 1$: random, no pressure
- $k = 2$–3: mild (default)
- large k: fast, loses diversity

Gaussian mutation: each gene, with probability p_m, gets $+\,\mathcal{N}(0, \sigma_j^2)$ with $\sigma_j = 0.1\,(u_j - \ell_j)$

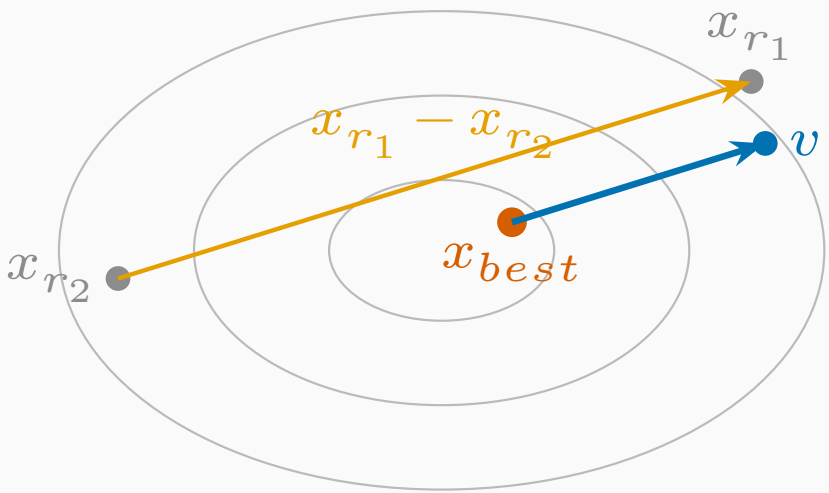
Blend crossover (BLX-α)



$$c_j \sim \mathcal{U}\big(\,\min(p_{1j}, p_{2j}) - \alpha d_j,\ \text{ma}$$

with $d_j = |p_{1j} - p_{2j}|$ and $\alpha = 0.5$: children may land **outside** the parents, which keeps the population from shrinking too fast

Differential Evolution



1. **Mutant:** $v = x_{best} + F\left(x_{r_1} - x_{r_2}\right)$ (variant `best/1`; `rand/1` uses x_{r_3} as the base)
2. **Crossover:** each gene of the trial comes from v with probability CR, else from the parent
3. **Selection:** the trial replaces its parent only if it is **better** (greedy)

- The step size comes from the **population's own spread**: large while scattered, small once converged

DE Variants and Adaptive DE

Variant	Mutant	Behaviour
`rand/1`	$x_{r_3} + F(x_{r_1} - x_{r_2})$	diverse
`best/1`	$x_{best} + F(x_{r_1} - x_{r_2})$	greedy, can stall
`c-to-pbest/1`	$x + F(x_{pb} - x) + F(x_{r_1} - x_{r_2})$	towards top $p\%$

- **JADE**, **SHADE**: learn F and CR from the settings that produced survivors (a memory of h settings in SHADE)
- pyBlindOpt: `differential_evolution(f, bounds, variant="current-to-pbest/1/bin", policy="shade")`
- pyBlindOpt's own benchmark ($d = 10$, 25 seeds): on Rastrigin, `best/1` has median 7.96 and SHADE 0.00015

Grey Wolf Optimization



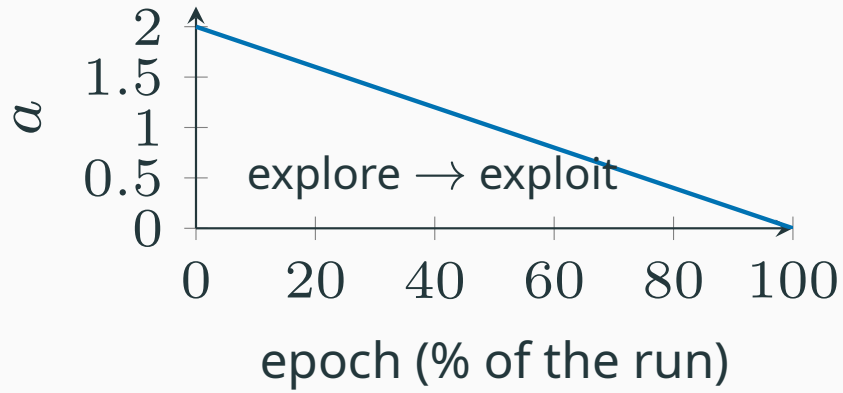
- The three best wolves **estimate** where the prey (the optimum) is
- Every wolf moves to a random point **around** that estimate
- a falls linearly from 2 to 0: $|A| > 1$ **explores** (overshoots the prey), $|A| < 1$ **exploits**

$$X_{t+1} = X_{prey} - A \odot |C \odot X_{prey} - X_t|$$

with $A \sim \mathcal{U}(-a, a)$ and $C \sim \mathcal{U}(0, 2)$ drawn per coordinate

Enhanced Grey Wolf (EGWO)

$$X_{prey} = w_1 X_\alpha + w_2 X_\beta + w_3 X_\delta + \mathcal{N}(0, \sigma^2), \quad w_1 \geq w_2 \geq w_3$$



- The prey is a **weighted** mix of the leaders (the alpha counts most), plus an estimation error
- σ is proportional to how much the three leaders **disagree**: wide while they are scattered, zero once they agree
- Reference: Luo (2019), *Applied Soft Computing* 77

pyBlindOpt in Practice

```python
import numpy as np, pyBlindOpt as pbo
from pyBlindOpt import init, utils

bounds = np.array([[-5.0, 5.0]] * 10)                # 10 dimensions
rng = np.random.default_rng(42)
pop = init.oblesa(f, bounds, n_pop=20, seed=rng)   # initialization
best_x, best_f = pbo.differential_evolution(
    f, bounds, population=pop, n_iter=100, seed=rng,
    variant="current-to-pbest/1/bin", policy="shade")
```

- `f` takes a point or a population: write it with `x[..., i]`
- Same signature for `genetic_algorithm`, `enhanced_grey_wolf_optimization`, `hill_climbing`, ...
- Class interface (`pbo.DifferentialEvolution(...)`) + callbacks to record every population

Live Demo B3: Populations at Work

- Notebook `02_blind_optimization.ipynb`, section **B3**
- GA, DE and EGWO on a shifted **Sphere** and a shifted **Rastrigin**: snapshots at epochs 0, 3, 10 and 30
- Watch how each population **contracts**: GA by recombination, DE by shrinking difference vectors, EGWO by shrinking a
- **pyOptViewer:** the same algorithms animated live, with the surface each one has explored

```
git clone https://github.com/mariolpantunes/pyOptViewer
cd pyOptViewer && python -m venv venv && venv/bin/pip install .
venv/bin/python -m optviewer    # http://127.0.0.1:8000
```

Comparing Stochastic Optimizers

Rules for a Fair Comparison

same budget
in evaluations,
not epochs

many seeds
one run is
an anecdote

spread
medians,
quartiles,
box plots

shifted
problems,
not centred

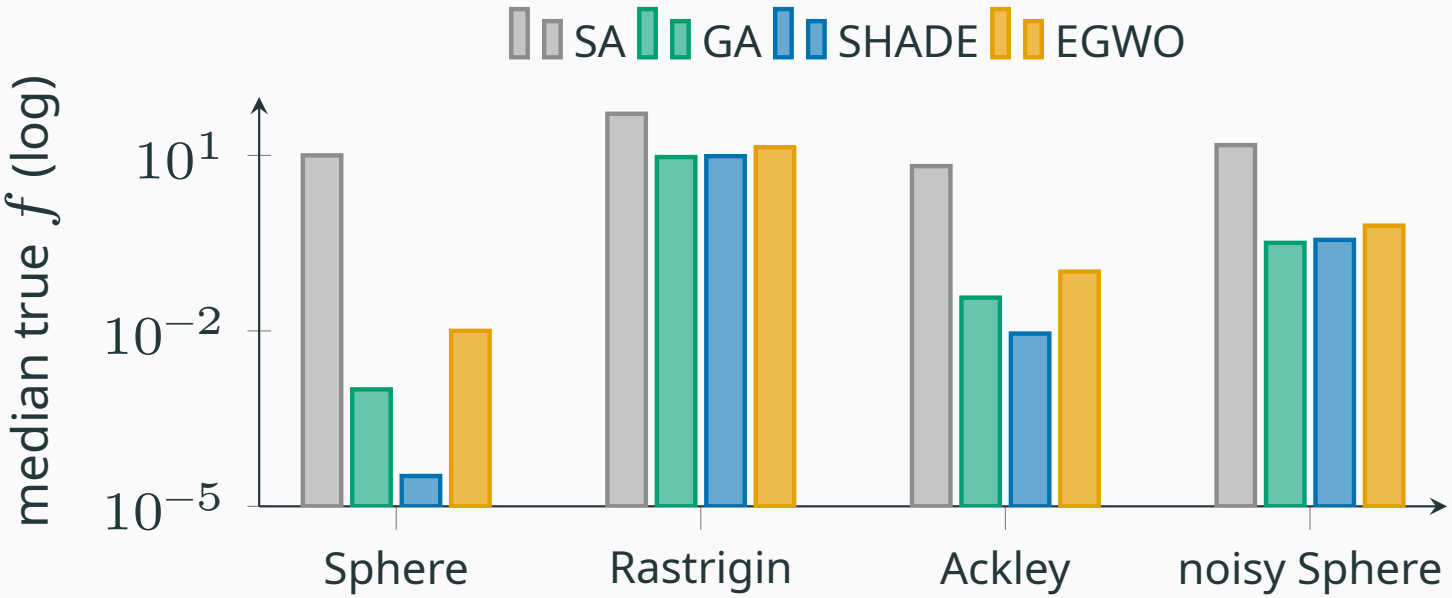
- The same statistics as for models in Class 01: a stochastic optimizer is an experiment
- Under **noise**, report the *true* f of the returned point: the best *observed* value is optimistically biased (a lucky evaluation wins)
- Tune every contender with the same effort, or none of them

Seven Optimizers, Four Problems

Median true f	Sphere	Rastrigin	Ackley	Noisy Sphere
Random	18.05	85.65	6.20	18.05
Hill climb.	10.31	52.87	6.36	15.43
Sim. anneal.	10.01	51.56	6.56	15.00
GA	0.001	9.37	0.037	**0.32**
DE (`best/1`)	0.015	18.07	1.65	0.98
DE (SHADE)	**3e-5**	9.69	**0.009**	0.36
EGWO	0.010	13.78	0.103	0.63

- 15 seeds, shifted functions, $d = 10$, 20 individuals $\times$ 100 epochs ($\approx$ 2000 evaluations); lower is better
- In 10D, single-point methods are far behind every population method
- GA wins on Rastrigin by a small margin: **no algorithm wins everywhere**

No Free Lunch, Again



- The ranking changes with the landscape (No Free Lunch theorem, Class 01)
- Greedy methods (DE `best/1`) are excellent on some seeds and stuck on others: look at the **spread**
- On the noisy Sphere, GA and SHADE are close: noise blurs the differences

Best of Both Worlds: Hybrids

seed	DE (48 evaluations) x	f	+ 50 GD steps x	f
0	$+1.530$	-0.795	$+1.557$	-0.811
1	-0.647	-1.070	-0.670	-1.081
2	-0.629	-1.049	-0.670	-1.081
3	-0.669	-1.081	-0.670	-1.081
4	$+1.561$	-0.810	$+1.557$	-0.811

- f_{rugged}: the **blind** stage chooses the basin, the **gradient** stage reaches its bottom cheaply
- Seeds 0 and 4 chose the second-best basin: 48 evaluations are not enough to be sure
- Blind global search + local gradient refinement = **memetic** algorithm

Live Demo B4–B5: Comparing and Combining

- Notebook `02_blind_optimization.ipynb`, sections **B4** and **B5**
- 420 runs: 7 optimizers $\times$ 4 problems $\times$ 15 seeds, as a table (polars) and box plots
- The noisy Sphere: why we evaluate the returned point on the *true* function
- DE followed by gradient descent with `jax.grad`

Summary

Choosing an Optimizer

Situation	First choice
Differentiable loss, many parameters (ML training)	Adam or SGD + momentum (via JAX / Keras)
Differentiable, few parameters, smooth	Newton or L-BFGS
Convex	any gradient method: local = global
No gradient, few dimensions, cheap f	DE (SHADE), GA, EGWO
Multimodal, gradient available	population search + gradient polish (hybrid)
Expensive f (e.g. hyperparameters)	few, well-placed evaluations: good initialization

Gradient-Based versus Blind

	Gradient-based	Blind (population)
Needs	∇f (JAX gives it for free)	only $f(x)$
Scales to	millions of parameters	tens to hundreds
Finds	a local minimum, fast	a good basin, slowly
Cost per step	one gradient ($\approx$ a few f)	N evaluations
Hyperparameters	learning rate (β's)	N, operators, rates
Weak on	plateaus, steps, noise, many basins	precision, high dimension

Key Takeaways

- **Training is optimization:** $\theta^\star = \arg\min \mathcal{L}(\theta)$, and the loss is the code length of Class 01
- **Gradients** are the most efficient signal when they exist; the learning rate and the curvature decide the speed
- **Newton** uses curvature: few steps, expensive, fooled by maxima and saddles; **Adam** is a cheap diagonal imitation
- **JAX** derives gradients of any model and loss, even through a simulation
- **Blind** methods need only values: they handle black boxes, steps, noise and many basins, at the cost of evaluations
- **Initialization** matters most with small budgets; always test on shifted problems
- **Compare** stochastic optimizers like models: equal budgets, many seeds, spreads